EXP.NO: 5	Write a Servlet to demonstrate the difference between HTTP GET and POST methods by creating a form and handling requests accordingly.
DATE:05/03/2025	

AIM:

To demonstrate the difference between the HTTP GET and POST methods by creating a form in an HTML page and handling the form submissions using two different HTTP methods in a Java Servlet.

ALGORITHM:

1. **Input:**

- The user fills out a web form with their name and email.
- The form will use both GET and POST methods for data submission.

2. **Steps for Creating and Handling the HTTP GET and POST Requests:**

1. **Create an HTML Form:**

    - Design an HTML form with two input fields: one for the name and one for the email.
    - Create two buttons to allow the user to submit the form using GET or POST methods.
    - Use the action attribute to define the Servlet URL and specify the form method (either GET or POST).

2. **Create the Servlet to Handle GET and POST Requests:**

    - Write a Java Servlet class that will handle both GET and POST requests.
    - The servlet will have two methods:
    - doGet() to handle the GET requests.
    - doPost() to handle the POST requests.
    - In both methods, retrieve the form data using request.getParameter().

- In the doGet() method, process and display the data in the URL.
- In the doPost() method, process and display the data in the HTTP request body.

3. Handle GET Requests in the Servlet:

- In the doGet() method, use request.getParameter() to retrieve the name and email submitted via GET.
- Output the data as a response, including the name and email in the URL (visible to the user).

4. Handle POST Requests in the Servlet:

- In the doPost() method, use request.getParameter() to retrieve the name and email submitted via POST.
- Output the data as a response, displaying the name and email without exposing them in the URL (more secure).

5. Return the Output:

- The Servlet should return an HTML page displaying the name and email entered by the user.
- For the GET method, the output should be visible in the URL.
- For the POST method, the output should be visible in the response body, not the URL.

6. Deploy and Test the Servlet:

- Compile the Servlet class and deploy it to a Servlet container (like Apache Tomcat).
- Test the form using GET and POST methods to see the difference in data submission.

7. Observe and Compare GET and POST Results:

- Test the GET method: The form data is sent in the URL.
- Test the POST method: The form data is sent in the HTTP request body, not visible in the URL.

PROGRAM:

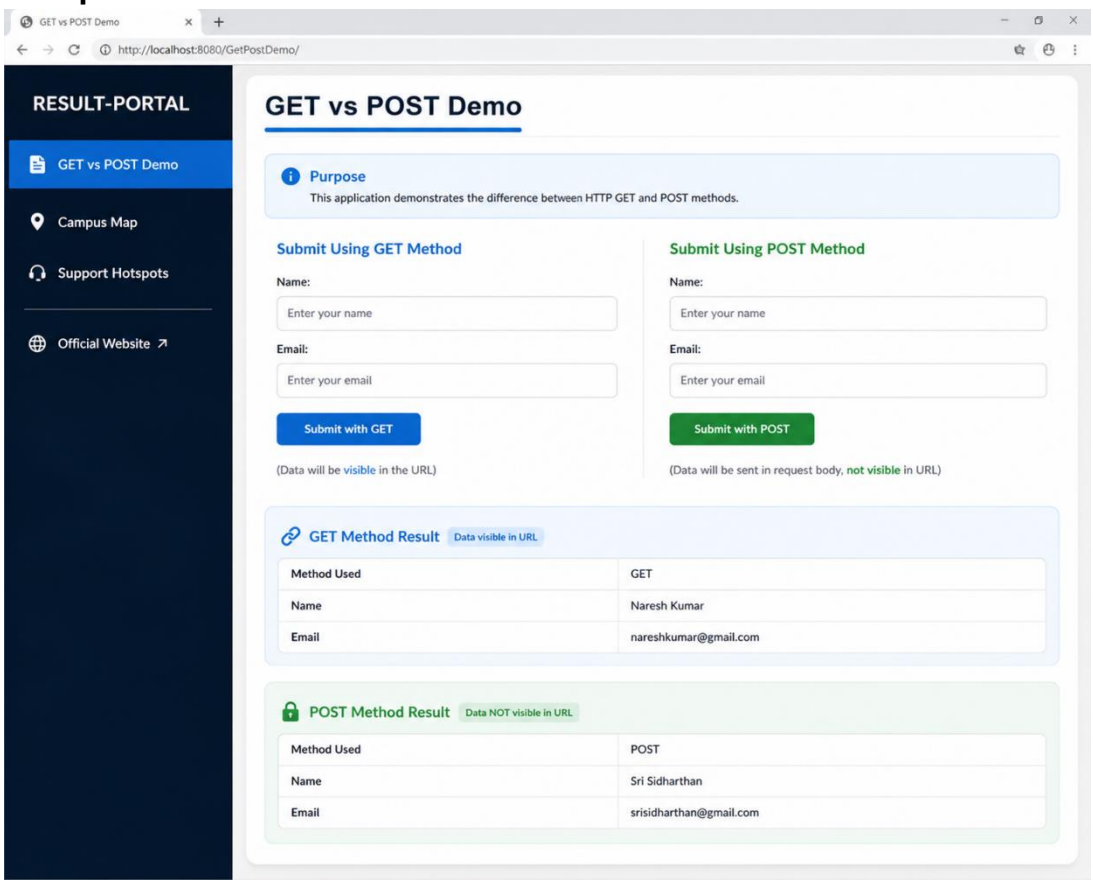
```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>GET vs POST Demo</title>

<style>
:root {
    --sidebar-color: #2c3e50;
    --accent: #3498db;
    --success: #27ae60;
}

body {
    font-family: 'Segoe UI';
    margin: 0;
    display: flex;
    height: 100vh;
    background-color: #eef2f3;
}

nav {
    width: 240px;
    background-color: var(--sidebar-color);
    color: white;
    padding-top: 20px;
}

.nav-link {
    display: block;
    padding: 15px 25px;
    color: #bdc3c7;
    cursor: pointer;
}

.nav-link:hover {
    background-color: #34495e;
    color: white;
}

main {
```

```
    flex: 1;
    padding: 30px;
}

.container {
    background: white;
    max-width: 700px;
    margin: auto;
    padding: 30px;
    border-radius: 12px;
}

input {
    width: 100%;
    padding: 10px;
    margin: 8px 0;
}

button {
    width: 48%;
    padding: 10px;
    margin: 5px 1%;
    background: var(--accent);
    color: white;
    border: none;
}

.result-box {
    margin-top: 20px;
    padding: 15px;
    border-radius: 8px;
    background: #f8f9fa;
}
</style>
</head>

<body>

<nav>
    <h2 style="text-align:center;">RESULT-PORTAL</h2>
    <div class="nav-link">GET vs POST Demo</div>
</nav>
```

```
<main>
<div class="container">

<h1>GET vs POST Demonstration</h1>

<!-- GET FORM -->
<form method="get">
    <input type="text" name="name" placeholder="Enter Name" required>
    <input type="email" name="email" placeholder="Enter Email" required>
    <button type="submit">Submit (GET)</button>
</form>

<!-- POST FORM -->
<form method="post">
    <input type="text" name="name" placeholder="Enter Name" required>
    <input type="email" name="email" placeholder="Enter Email" required>
    <button type="submit">Submit (POST)</button>
</form>

<div class="result-box">
<script>
const params = new URLSearchParams(window.location.search);
const name = params.get("name");
const email = params.get("email");

if(name && email){
    document.write("<h3>GET Method Used</h3>");
    document.write("<p>Name: " + name + "</p>");
    document.write("<p>Email: " + email + "</p>");
    document.write("<p>Data is visible in URL</p>");
}
</script>

<?php
if($_SERVER["REQUEST_METHOD"] == "POST"){
    $name = $_POST["name"];
    $email = $_POST["email"];

    echo "<h3>POST Method Used</h3>";
    echo "<p>Name: $name</p>";
    echo "<p>Email: $email</p>";
    echo "<p>Data is NOT visible in URL</p>";
}
```

```
?>
</div>

</div>
</main>

</body>
</html>
```

Output:



RESULT:

A Servlet is created to demonstrate the difference between HTTP GET and POST methods by creating a form and handling requests accordingly.